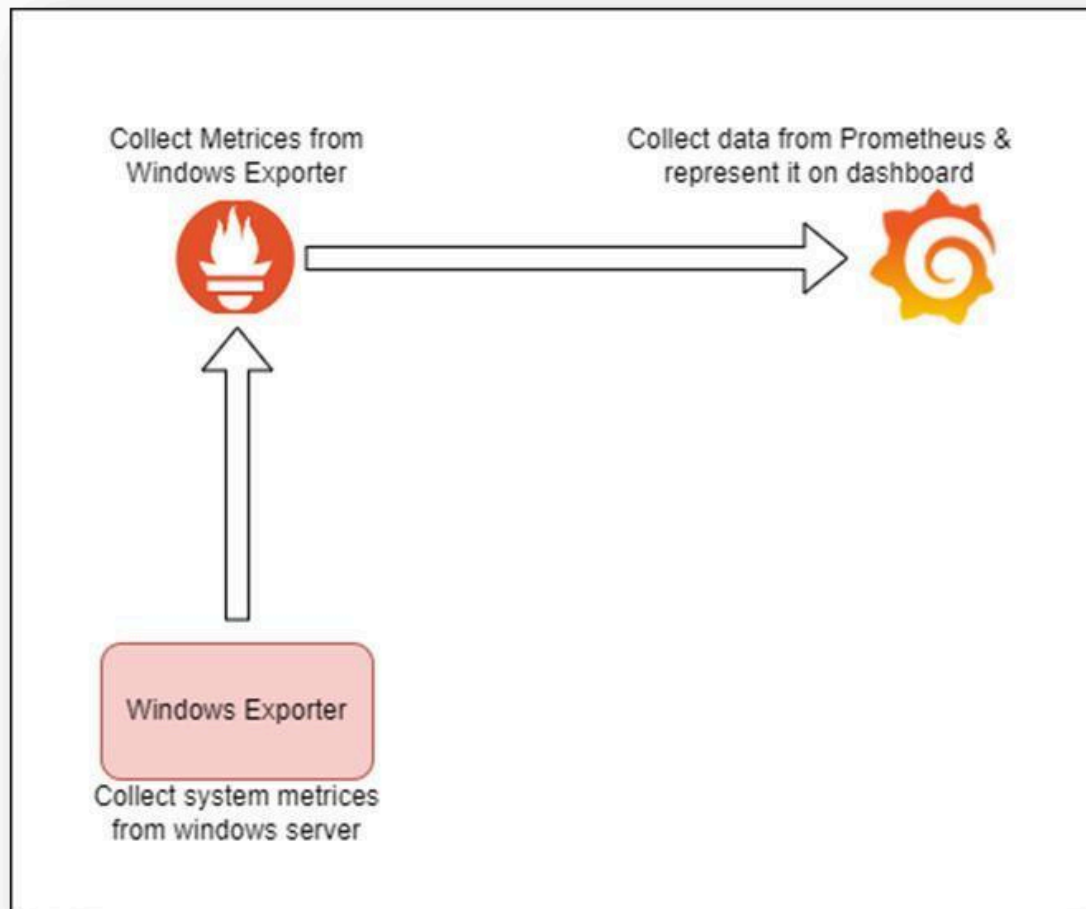


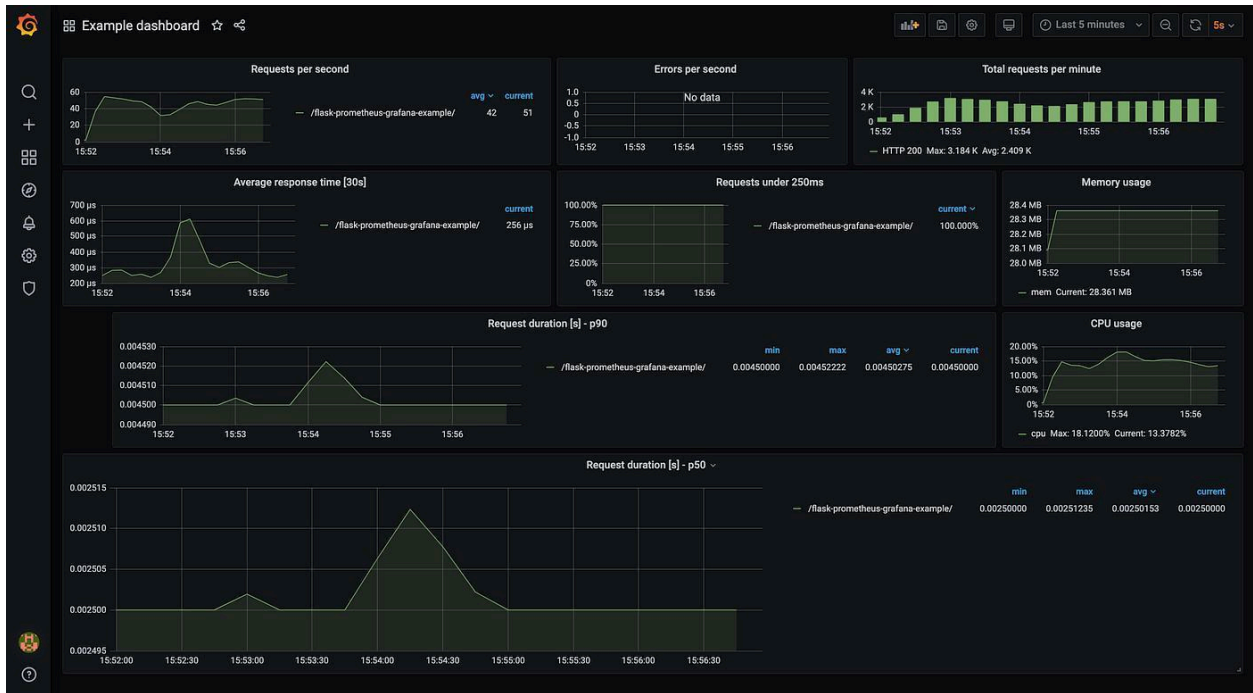
Yes. If you are doing this **directly on Windows**, don't use the Ubuntu/Linux commands I gave earlier. We can do the entire practical using **Windows CMD/PowerShell**, without Docker or a VM.

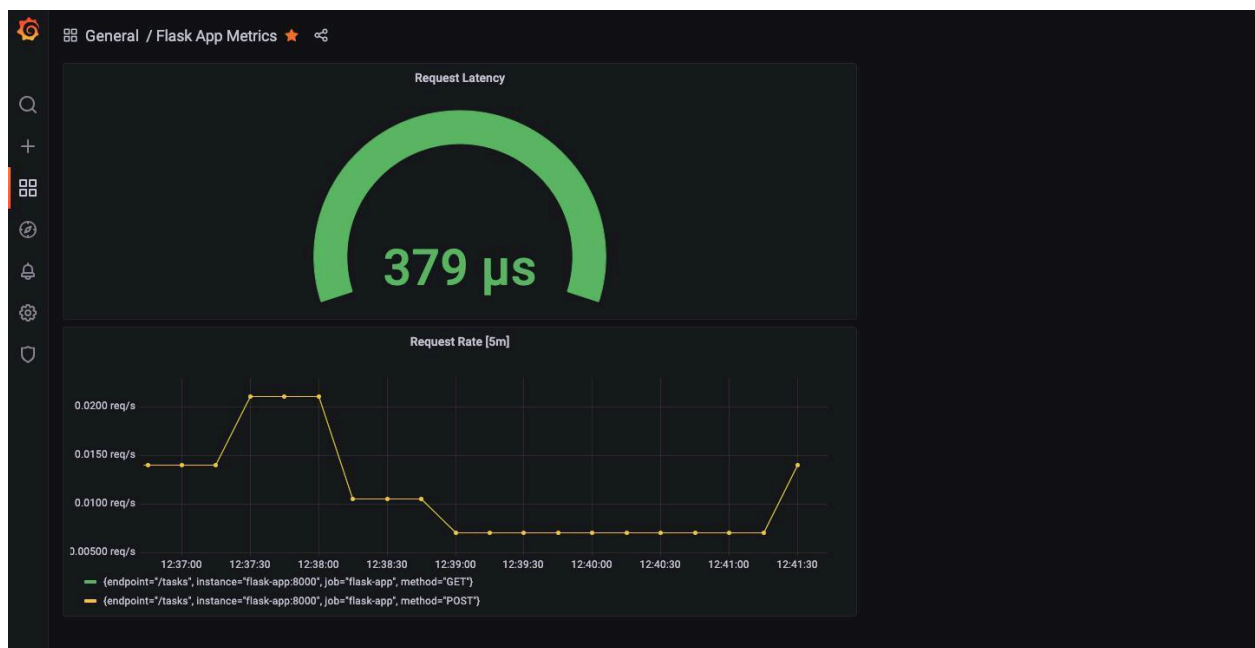
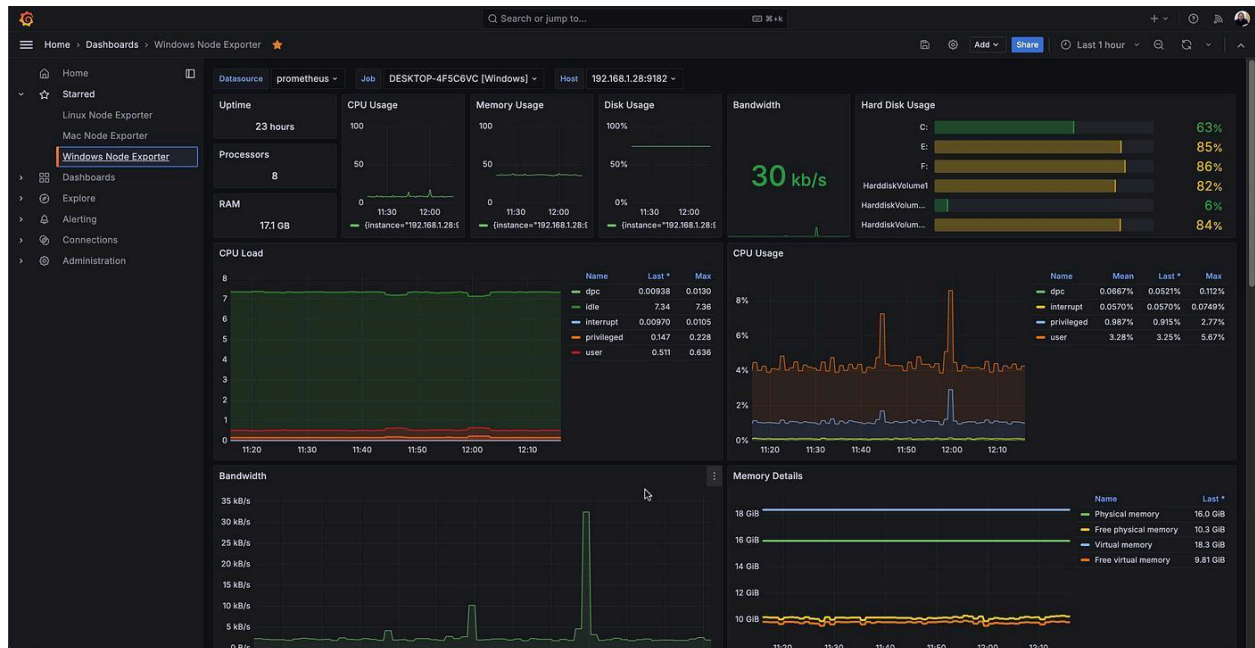
For your practical, I recommend this setup:

Python Flask → Prometheus → Windows Exporter → Grafana

Prometheus provides a Windows windows-amd64 binary, and Grafana officially supports Windows. ([Prometheus](#))







Practical: Monitoring and Observability with Prometheus and Grafana on Windows

What you will install

Software	Purpose	Port
Python + Flask	Application	5000
Python Prometheus Client	Exposes application metrics	8000

Software	Purpose	Port
Prometheus	Collects/stores metrics	9090
Windows Exporter	CPU & memory metrics	9182
Grafana	Creates dashboards	3000

Part A — Set up Prometheus

Step 1: Download Prometheus for Windows

Go to the official Prometheus download page:

[Prometheus Downloads](#)

For a normal 64-bit Windows computer, download:

prometheus-3.14.0.windows-amd64.zip

The current stable release listed by Prometheus is 3.14.0; there is also a newer release candidate, so use the stable release for your practical. ([Prometheus](#))

Extract it, for example:

C:\prometheus

You should see:

C:\prometheus

|

├── prometheus.exe

├── promtool.exe

├── prometheus.yml

└── consoles

Step 2 — Configure Prometheus

Open:

C:\prometheus\prometheus.yml

Open it with Notepad and **replace its contents** with:

global:

scrape_interval: 5s

scrape_configs:

- job_name: "prometheus"

static_configs:

- targets: ["localhost:9090"]

- job_name: "python-app"

static_configs:

- targets: ["localhost:8000"]

- job_name: "windows"

static_configs:

- targets: ["localhost:9182"]

Save the file.

The important part is:

- job_name: "python-app"

static_configs:

- targets: ["localhost:8000"]

This tells Prometheus:

Collect metrics from my Python application's metrics endpoint.

Prometheus works by periodically scraping HTTP metric endpoints from monitored targets.

([Prometheus](#))

Part B — Create the Python Application

Step 3: Create a project folder

Open **CMD**.

```
mkdir C:\monitoring-app
```

```
cd C:\monitoring-app
```

Create a virtual environment:

```
python -m venv venv
```

Activate it:

```
venv\Scripts\activate
```

You should see:

```
(venv) C:\monitoring-app>
```

Step 4: Install Flask and Prometheus Client

Run:

```
pip install flask prometheus-client
```

The official Prometheus Python client is installed with `pip install prometheus-client`.

([Prometheus](#))

Check:

```
pip list
```

You should see:

```
Flask
```

```
prometheus-client
```

Step 5: Create app.py

Create:

C:\monitoring-app\app.py

Put this code inside:

```
from flask import Flask
```

```
from prometheus_client import Counter, start_http_server
```

```
app = Flask(__name__)
```

```
# Count API requests
```

```
REQUEST_COUNT = Counter(
```

```
    "api_requests_total",
```

```
    "Total number of API requests"
```

```
)
```

```
@app.route("/")
```

```
def home():
```

```
    REQUEST_COUNT.inc()
```

```
    return "Hello! Flask application is running."
```

```
@app.route("/hello")
```

```
def hello():
```

```
    REQUEST_COUNT.inc()
```

```
    return "Hello from the monitoring application!"
```

```
if __name__ == "__main__":  
  
    # Prometheus metrics server  
    start_http_server(8000)  
  
    # Flask application  
    app.run(host="0.0.0.0", port=5000)
```

Step 6: Run the Python Application

In CMD:

```
cd C:\monitoring-app  
venv\Scripts\activate  
python app.py
```

You should see something similar to:

* Running on http://127.0.0.1:5000

Keep this CMD window open.

Step 7: Test Flask

Open your browser:

http://localhost:5000

You should see:

Hello! Flask application is running.

Also test:

http://localhost:5000/hello

Step 8: Test Prometheus Metrics

Open:

`http://localhost:8000`

You should see Prometheus metrics.

Look for:

`api_requests_total`

For example:

`api_requests_total 2.0`

The Python client exposes metrics through an HTTP endpoint; `start_http_server(8000)` is the simple built-in approach. ([Prometheus](#))

Part C — Start Prometheus

Step 9: Open a Second CMD

Keep your Flask application running.

Open another **CMD** window.

Run:

`cd C:\prometheus`

Then:

`prometheus.exe --config.file=prometheus.yml`

You should see Prometheus starting.

Do not close this CMD window.

Step 10: Open Prometheus

Go to:

`http://localhost:9090`

You should see the Prometheus interface.

Go to:

Status

→ Targets

You should see:

prometheus UP

python-app UP

windows UP

Initially, windows will not work until we install Windows Exporter.

Part D — Install Windows Exporter

This is important because you specifically want:

- CPU usage
- Memory consumption

For Windows, use **windows_exporter** rather than Linux Node Exporter. The project provides collectors for Windows CPU, memory, logical disks, network and other system metrics. ([GitHub](#))

Step 11: Download Windows Exporter

Official project:

[windows_exporter](#)

Download the Windows installer from the Releases section.

Install the **64-bit Windows version**.

After installation, Windows Exporter normally exposes metrics on:

`http://localhost:9182/metrics`

Step 12: Test Windows Exporter

Open your browser:

`http://localhost:9182/metrics`

You should see metrics such as:

windows_cpu_...

windows_memory_...

windows_logical_disk_...

The Windows exporter has CPU and logical-disk collectors enabled by default, with memory available as a collector as well. ([GitHub](#))

Step 13: Check Prometheus Again

Go to:

<http://localhost:9090>

Then:

Status

→ Targets

You should now have:

prometheus UP

python-app UP

windows UP

This means:

Prometheus is successfully collecting data from your Windows computer and Python application.

Part E — Install Grafana

Step 14: Download Grafana

Use the official Grafana download page:

[Grafana Windows Download](#)

Choose:

Windows

→ Windows Installer

→ 64 Bit

Run the .msi installer.

Grafana officially supports Windows and provides a Windows 64-bit installer. ([Grafana Labs](#))

Step 15: Start Grafana

After installation, open:

`http://localhost:3000`

You should see the Grafana login page.

Log in using the administrator account you created during installation.

Part F — Connect Grafana to Prometheus

Step 16: Add Prometheus Data Source

In Grafana:

Connections



Data sources



Add data source



Prometheus

For URL enter:

`http://localhost:9090`

Click:

Save & Test

You should receive a successful connection message.

Grafana has built-in Prometheus data-source support. ([Grafana Labs](#))

Part G — Create Dashboard

Go to:

Dashboards

→ New

→ New Dashboard

→ Add visualization

Select:

Prometheus

as the data source.

Now we can create three important panels.

Panel 1 — API Request Rate

Use this PromQL query:

```
rate(api_requests_total[1m])
```

Set visualization:

Time series

Panel title:

API Request Rate

This displays approximately how many requests your Flask application receives per second.

Panel 2 — CPU Usage

Because you are on Windows, use Windows Exporter metrics.

First, in the Prometheus query box, search for:

```
windows_cpu
```

You can inspect the exact metric names available from your installed exporter.

A common CPU utilization query is:

```
100 - (  
  100 *  
  avg by (instance) (  
    rate(windows_cpu_time_total{mode="idle"}[5m])  
  )  
)
```

Set:

Visualization: Gauge

Title:

CPU Usage %

Panel 3 — Memory Usage

Search in Prometheus for:

windows_memory

A commonly used query is:

```
100 *  
(  
  1 -  
  windows_memory_available_bytes  
  /  
  windows_memory_physical_total_bytes  
)
```

Set:

Visualization: Gauge

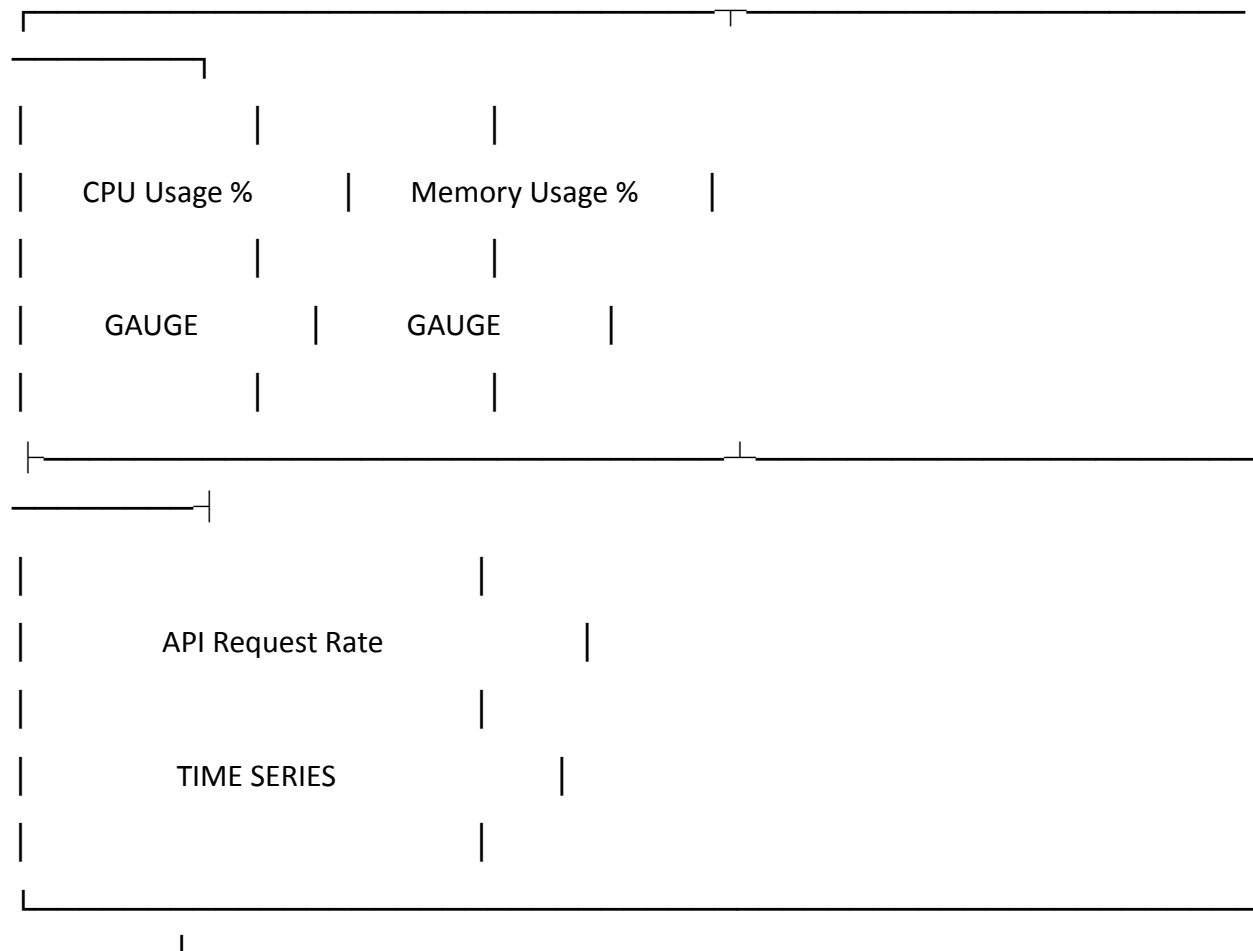
Title:

Memory Usage %

Note: Metric names can vary slightly with the installed windows_exporter version/configuration. If the query returns no data, open <http://localhost:9182/metrics> and use the exact memory metric names shown there.

Your Final Grafana Dashboard

Create three panels:



Step 17 — Generate API Requests

Go back to your browser:

<http://localhost:5000/>

Refresh the page several times.

Also visit:

`http://localhost:5000/hello`

multiple times.

Or use CMD:

```
curl http://localhost:5000/
```

Run it several times.

Then Grafana's:

API Request Rate

panel will show the activity.

Complete Windows Setup

You will have **4 CMD windows/processes**:

CMD 1 — Python

```
cd C:\monitoring-app
```

```
venv\Scripts\activate
```

```
python app.py
```

CMD 2 — Prometheus

```
cd C:\prometheus
```

```
prometheus.exe --config.file=prometheus.yml
```

Windows Exporter

Runs as a **Windows service** after installation.

Grafana

Runs as a **Windows service** after installation.

Then access:

Python:

http://localhost:5000

Python Metrics:

http://localhost:8000

Windows Metrics:

http://localhost:9182/metrics

Prometheus:

http://localhost:9090

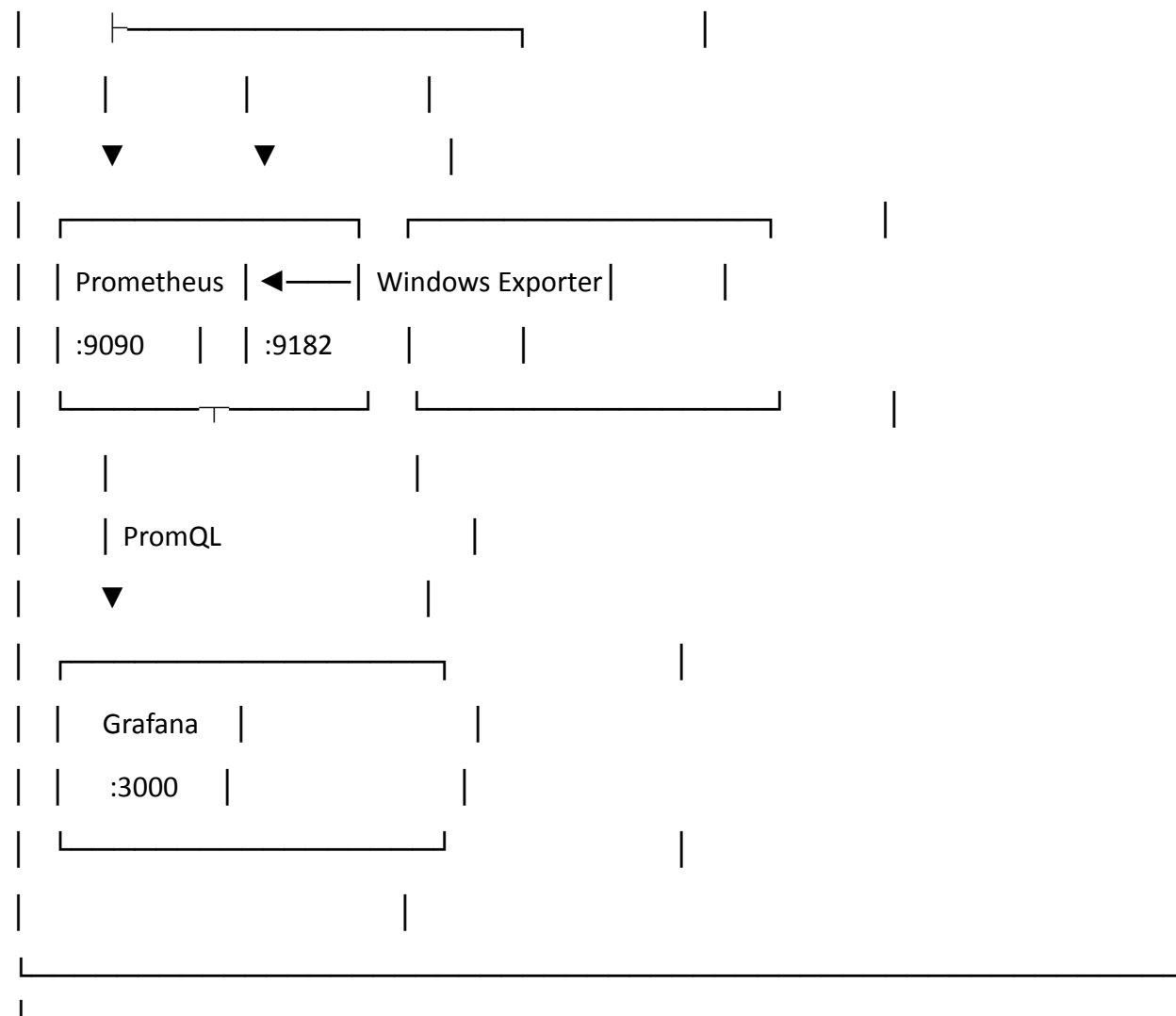
Grafana:

http://localhost:3000

Final architecture

WINDOWS PC





For your college practical, this Windows setup is enough—you do not need Ubuntu, VirtualBox, Docker, or a VM.